

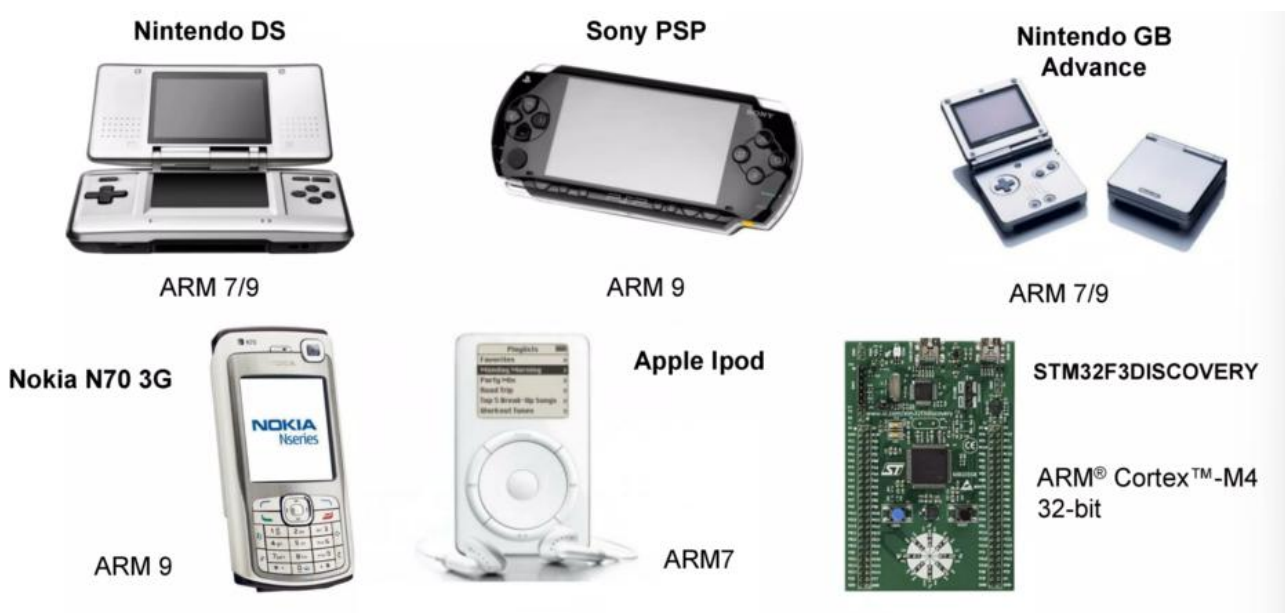
Sommario

1. I processori ARM	1
1.1 Intellectual Property core	2
1.2 Modello di Programmazione	3
1.2 Current Programm status register	3
1.3 Modalità di privilegio	4
1.4 Stato Thumb	4
1.5 ARM7	5
1.6 ARM9	5
1.7 Versioni Cortex A e Cortex-M	6
1.8 Trust Zone	6
1.9 Virtualization Extension	7
2. Overview dei sistemi cloud, IoT, edge, e fog	9

1. I processori ARM

ARM è una storia di tecnologie, ci permette di capire cosa succede nell'evoluzione tecnologica dei sistemi. Fu sviluppata nel 1985 e **introdusse il primo processore RISC nel 1987**. **Le prime due versioni sono a 26 bit**, mentre **dalla quarta in poi sono a 32 bit introducendo anche registri di stato separati**. La versione 4 è ancora supportata.

Arm è utilizzato in moltissimi dispositivi



È utilizzato anche nello schedario ST dei progetti.

Problematiche di progettazione:

- Arm è un Soc. sistema su circuito integrato. In un solo chip contiene un intero sistema, ossia integra oltre al processore centrale anche un chipset ed eventualmente altri controller, circuiteria I/O. Non abbiamo come l'FPGA tanti oggetti e lo configuriamo, ma questo sistema è mettere sul chip solamente quello che effettivamente serve. Negli ultimi anni il numero di porte di un SoC continua ad aumentare e non è facile produrre sistemi completamente testati con questa complessità. Si è pensato quindi di basare una parte del progetto su componenti già esistenti in modo da ridurre i tempi di progettazione. Come progettare un sistema sfruttando parti già esistenti, provenienti da varie fonti?
- Come essere sicuri che il sistema rispetti i requisiti di performance richiesti?
- Come può un progettista assicurarsi dei risultati ottenuti prima che il chip sia disponibile?

1.1 Intellectual Property core

È una parte di logica riusabile, una standard o un chip layout di cui qualcuno possiede la proprietà intellettuale. Può essere data in licenza per essere usata come componente di base attorno al quale viene costruito uno specifico ASIC/FPGA.

Tipicamente un produttore impiega un ARM core attorno al quale viene progettata la logica addizionale per un dispositivo.

L'IP Core è come l'equivalente hardware di una libreria software pre-compilata. I progettisti acquistano o ottengono in licenza blocchi funzionali già pronti, testati e ottimizzati.

L'ARM non produce chip fisici, ma vende IP core. Un'azienda terza prende il core ARM, lo usa come "cervello" centrale e ci costruisce attorno la propria logica personalizzata per creare un chip unico.

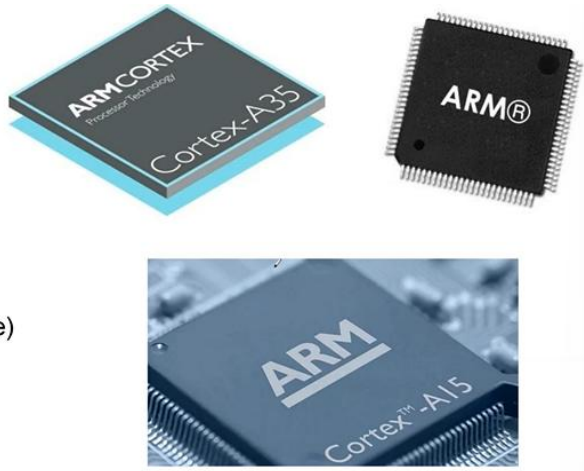
3 tipi di licenza:

- Perpetua: paghi una volta e ottieni il diritto di usare quel blocco per progettare e produrre chip per sempre, senza limiti di tempo.
- A termine: hai un periodo di tempo per completare la fase di progettazione. Una volta che il chip è pronto, mantieni il diritto di produrlo a tempo indeterminato, ma scaduto il termine non potrai iniziare a progettare nuovi chip con quell'IP core senza rinnovare il contratto. Finito quel tempo puoi continuare a produrre i chip già progettati, ma non puoi progettarne di nuovi.
- A prodotto: è la licenza più restrittiva ma spesso più economica. Puoi usare l'ip core solo per sviluppare uno specifico dispositivo. Se un giorno decidessi di fare una versione nuova non puoi.

A seconda delle lettere che ho dopo il nome arm posso capire come è fatto il processore. Ad esempio x significa serie, y se c'è MMU etc.

ARMxyzTDMIEJFS:

- x: serie
- y: MMU
- z: cache
- T: Thumb
- D: Debugger
- M: Moltiplicatore
- I: ICE Macrocell integrato
- E: istruzioni migliorate
- J: accelerazione Java (Jazelle)
- F: Vector Floating-point
- S: versione sintetizzabile



1.2 Modello di Programmazione

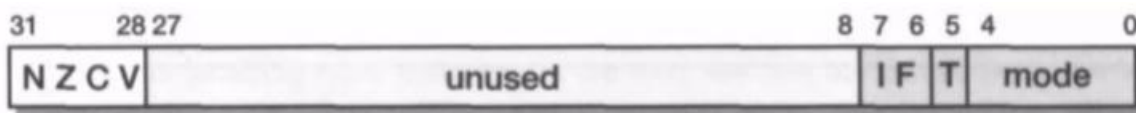
I registri sono general-purpose e special-purpose. Il numero di registri varia a seconda della proprietà di privilegio. In generale i registri principali sono 16 nominati da R0 e R15:

- R0-R12 sono registri di uso generale
- R13 o SP è il registro dello stack Pointer, cioè l'indirizzo di memoria dello stack.
- R14 è il Link register dove si salva l'indirizzo di ritorno di una procedura
- R15 è il PC che contiene l'indirizzo di memoria della prossima istruzione da eseguire.

In totale possono avere 37 registri, con 3 registri aggiuntivi se sono implementate le estensioni di sicurezza, 3 in più se sono implementate le estensioni di virtualizzazione.

1.2 Current Program status register

Un registro fondamentale è il CPSR che tiene traccia dello stato del programma.



È un registro a 32 bit che contiene tutte le informazioni fondamentali sullo stato attuale della CPU in un dato istante.

I bit 31-28 APSR vengono aggiornati dopo le istruzioni matematiche o logiche e servono anche per i salti condizionati.

N: risultato negativo

Z: risultato zero

C: c'è stato un riporto

V: overflow

I bit 7-6 servono per le interruzioni. I se messo a 1 disabilita le interruzioni, F se messo a 1 disabilita i Fast interrupts.

I primi 4 bit non possono essere modificati in modalità utente. Questo registro è sempre accessibile in qualunque modo operi il processore. A valle di un'eccezione l'intero contenuto viene ricopiato in un altro registro per poi essere ripristinato al termine.

1.3 Modalità di privilegio

La maggior parte dei programmi viene eseguita in modalità Utente. Quando siamo in questa modalità, il programma non può accedere alle risorse di sistema e modificare la modalità stessa.

Le modalità diverse da quella utente vengono dette privilegiate. Esse hanno accesso alle risorse di sistema e possono cambiare modalità liberamente.

Processor Mode			Descrizione	CPSR[4:0]	Registri Accessibili
1	User	usr	Modalità di esecuzione dei programmi comuni	10000	R0 a R14, R15(PC), CPSR
2	FIQ	fiq	Gestione di Interrupt veloce, ad alta priorità	10001	R0 a R7, R8_fiq a R14_fiq, R15(PC), CPSR, SPSR_fiq
3	IRQ	irq	Gestione di Interrupt generico, priorità normale	10010	R0 a R12, R13_IRQ a R14_irq, R15(PC), CPSR, SPSR_irq
4	Supervisor	svc	Modalità privilegiata in cui si entra dopo un reset della CPU o a seguito dell'esecuzione di un'istruzione SVC	10011	R0 a R12, R13_SVC a R14_svc, R15(PC), CPSR, SPSR_svc
5	Abort	abt	Errore nell'accesso in memoria	10111	R0 a R12, R13_ABT a R14_abt, R15(PC), CPSR, SPSR_abt
6	Undefined	und	Istruzione Illegale	11011	R0 a R12, R13_und a R14_und, R15(PC), CPSR, SPSR_und
7	System	sys	Modalità privilegiata di esecuzione di un task del Sistema Operativo	11111	R0 a R14, R15(PC), CPSR

Nelle varie modalità si entra in seguito ad eventi e hanno registri aggiuntivi per evitare di corrompere lo stato utente quando si verificano eccezioni.

1.4 Stato Thumb

Molte funzioni complesse in un CISC richiedono una sola istruzione, mentre nei RISC possono richiedere più istruzioni. Per ridurre i costi di memoria di queste istruzioni aggiuntive vennero introdotti i processori con Thumb. Il set di istruzioni Thumb è costituito da istruzioni a 16 bit che fungono da abbreviazione per un sottoinsieme delle istruzioni a 32 bit dell'ARM standard.

Al contrario ogni istruzione THUMB può essere eseguita tramite l'equivalente a 32 bit.

Il bit 5 ovvero T se è 0 significa in stato ARM ovvero legge istruzioni standard a 32 bit, mentre se =1 è in stato Thumb cioè legge le istruzioni compresse a 16 bit ottime per risparmiare spazio in memoria.

Modi di indirizzamento dell'ARM

Nome alternativo	Esempio
Registro diretto	MOV R0, R1
Diretto	LDR R0, MEM
Immediato	MOV R0, #15 ADD R1, R2, #12
Registro indiretto	LDR R0, [R1]
Registro indiretto con offset	LDR R0, [R1, #4]
Registro indiretto preincremento	LDR R0, [R1, #4]!
Registro indiretto postincremento	LDR R0, [R1], #4
Registro indiretto Registro indicizzato	LDR R0, [R1, R2]
Registro indiretto indicizzato con scalatura	LDR R0, [R1, R2, LSL#2]
Relativo al Program Counter	LDR R0, [PC, #offset]

Il modo diretto opera con delle pseudoistruzioni.

1.5 ARM7

I processori ARM7 rilasciati per la prima volta nel 1993 sono stati i primi ad introdurre le istruzioni Thumb.

La pipe degli ARM7 è a 3 stadi:

- Fetch
- Decode
- Execute

Arm7 conteneva inoltre un'unica memoria per le istruzioni e per i dati. Questo quindi non consente di prelevare un'istruzione dalla memoria e al contempo utilizzarla per un'operazione di lettura e scrittura di un dato.

1.6 ARM9

La famiglia ARM9 è quella più diffusa. Adottano la struttura di tipo Harvard prevedendo due cache, una per i dati e una per le istruzioni. La pipeline è a 5 stadi

Dall'architettura ARM9TDMI la pipeline è diventata a 5 stadi:

- **Fetch:** l'istruzione è prelevata dalla memoria e messa nella pipeline;
- **Decode:** l'istruzione è decodificata e i registri operandi letti. Ci sono 3 ingressi di lettura per gli operandi in modo che la maggior parte delle istruzioni possano leggere gli operandi in un ciclo;
- **Execute:** l'operando è shiftato e viene generato il risultato dell'ALU. Se l'istruzione è load o store, l'indirizzo di memoria è calcolato nell'ALU;
- **Buffer/Data:** se richiesto viene effettuato l'accesso alla memoria dati. In caso contrario il risultato della ALU è semplicemente bufferizzato per un ciclo;
- **Write back:** il risultato generato dall'istruzione è scritto nel registro, incluso qualsiasi dato caricato dalla memoria.



1.7 Versioni Cortex A e Cortex-M

Questi processori sono orientati all'esecuzione di applicazioni utente basate su sistemi operativi per smartphone e Tv digitale.

La famiglia Cortex-M è stata progettata per microcontrollori e applicazioni embedded. Sono stati progettati per un minor consumo di area su silicio, una pipeline più corta e una minor frequenza di clock. In generale questa famiglia consente di avere un basso consumo energetico.

1.8 Trust Zone

Trust zone è un'estensione hardware dei processori ARM che si basa sull'idea di partizionare tutto l'hardware e il software in un SOC in due diversi modi: mondo sicuro e mondo normale.

Le barriere hardware implementate evitano che il mondo normale acceda alle risorse del mondo sicuro. Per esempio il codice eseguito nel mondo non sicuro non può accedere a zone di memoria allocate per il mondo sicuro. Nei sistemi tradizionali (senza TrustZone), la sicurezza è delegata interamente al Sistema Operativo (es. Linux, Android, Windows).

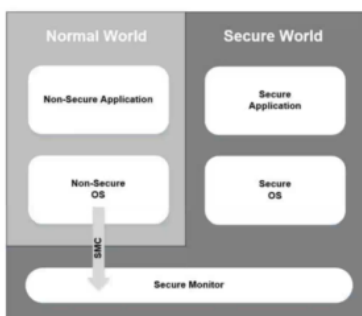


Fig. 1: TrustZone on ARM Cortex-A

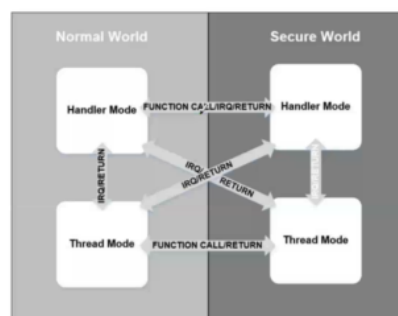


Fig. 2: TrustZone on ARM Cortex-M

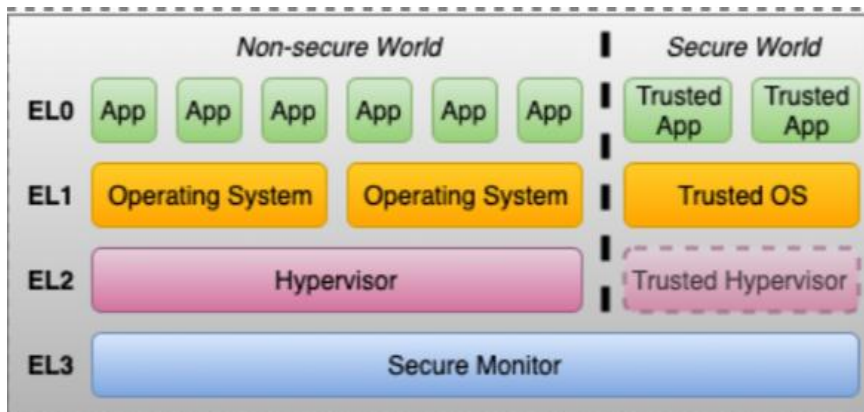
Il problema è che i Sistemi Operativi moderni sono enormi, composti da milioni di righe di codice. È statisticamente impossibile che non abbiano vulnerabilità. Se un attaccante riesce a ottenere i permessi di "root" (compromettendo il Kernel), il gioco è finito: ha accesso a tutta la memoria, a tutte le periferiche, alle chiavi crittografiche e alle password in chiaro. TrustZone nasce per rispondere a questa domanda: **Come possiamo mantenere dei dati al sicuro, anche se l'intero Sistema Operativo principale è stato completamente compromesso?**

1.9 Virtualization Extension

La virtualizzazione fornisce un modo per eseguire più sistemi operativi sullo stesso hardware. Consente il partizionamento delle risorse tra diverse VM impedendo a una VM di accedere a un'altra.

I sistemi operativi guest hanno l'illusione di possedere le risorse hardware tutte per se.

Il modello di virtualizzazione ARM si basa sul concetto di eccezione con priorità crescente. Le eccezioni possono essere intercettate dal livello a priorità superiore immediatamente successivo.



ad esempio il software in esecuzione su EL1 può eseguire una trap su EL2 utilizzando le istruzioni di HyperVisor Call. Il software in esecuzione su EL2 può eseguire una trap su EL3 utilizzando istruzioni Secur Monitor Call.

Esistono 4 livelli di eccezione:

- Spazio utente
- Livello del sistema operativo
- Hypervisor
- Monitor e il firmware sicuri.

Inoltre abbiamo la trust zone e nel mondo sicuro non è supportato nulla per l'hypervisor. il passaggio da mondo sicuro a non sicuro avviene tramite un'istruzione Secure Monitor Call

la virtualizzazione è governata dal livello EL2. Mentre il sistema operativo ha il compito di gestire le applicazioni, l'hypervisor ha il compito di gestire i diversi sistemi operativi. La virtualizzazione hardware consente di far girare più sistemi operativi (guest OS) contemporaneamente sulla stessa CPU fisica, facendo credere a ciascuno di essi di avere il controllo esclusivo della macchina.

Se un Sistema Operativo a livello EL1 fosse eseguito direttamente sull'hardware senza un Hypervisor, avrebbe libero accesso a tutte le configurazioni critiche della CPU.

Quando l'OS ospite cerca di eseguire un'istruzione critica che modificherebbe lo stato globale della macchina, l'hardware ARM blocca l'operazione e genera un'eccezione (un *Trap*), passando il controllo all'Hypervisor in EL2. L'Hypervisor valuta la richiesta, la simula in modo sicuro solo per quel software specifico, e poi restituisce il controllo all'OS.

Cosa significa Soc?

È un circuito integrato che racchiude al suo interno tutti i componenti fondamentali di un computer o di un sistema elettronico (CPU, GPU, memoria, controller di I/O, interfacce radio, ecc.) su un'unica piastrina di silicio, ottimizzando spazio e consumi.

Cosa significa ipcore?

Intellectual Property Core. In elettronica digitale, è un blocco logico pre-progettato e riutilizzabile. Può essere un componente semplice (un timer) o complesso (un'intera CPU o controller USB). Viene venduto o concesso in licenza sotto forma di codice (Soft IP, es. VHDL/Verilog) o di layout fisico già pronto (Hard IP).

Perché uno sviluppo ipcore può essere utile nell'ambito della progettazione?

Perché **abbatte drasticamente i tempi e i costi di sviluppo** (Time-to-Market). Invece di progettare da zero ogni singolo componente di un SoC, gli ingegneri acquistano e assemblano IP Core già testati, ottimizzati e privi di bug, potendosi concentrare solo sull'integrazione e sulle funzionalità distintive del proprio prodotto.

Arm è un processore o una famiglia di processori?

È una **famiglia di architetture** (Instruction Set Architecture) basata sul paradigma RISC.

Perché sulla famiglia arm sarebbe possibile implementare un sistema di virtualizzazione a più livelli?

Perché le architetture ARM moderne integrano **supporto hardware nativo alla virtualizzazione**. Dispongono di specifici livelli di privilegio (Exception Levels, da EL0 a EL3 e supportano la virtualizzazione nel mondo normale, quelli più moderni anche in quello sicuro (TrustZone), permettendo di annidare hypervisor o di separare contesti virtualizzati con altissima sicurezza.

Importante sottolineare la differenza tra architettura e implementazione:

- L'architettura definisce come un processore si comporti ovvero il modello di programmazione, registri, istruzioni
- L'implementazione si riferisce a come l'architettura sia realizzata ad esempio ARM-A8 e ARM-A9 sono basati su architettura ARMv7-A ma il primo ha una pipe a 13 stadi, il secondo a 8 stadi.

Gestione delle eccezioni:

si utilizzano le interrupt vettorizzato: fa uso di una interrupt vector table, presuppone l'esistenza di un intermediario tra processori e sorgenti d'interruzione:

- GIC generic interrupt controller: arm-cortex A-R
- NVIC: arm-cortex M
- Quando arriva un'interruzione si salva il CPSR (registro di stato) e l'indirizzo di ritorno in due registri appositi
- Il processore abbandona la modalità in cui si trovava (ad esempio la modalità *User* non privilegiata) ed entra nella modalità specifica dell'eccezione. Questo garantisce i privilegi hardware necessari per gestire il problema o la periferica.
- Vengono disabilitate le interruzioni
- Il processore forza il suo Program Counter (PC) a puntare a uno degli indirizzi fissi mostrati nella colonna "Offset" della tabella.
- Viene eseguito l'exception handler

Exception Vectors, vector address, and modes		
Offset	Vector	Mode
0x00	Reset	Supervisor
0x04	Undefined Instruction	Undefined
0x08	Supervisor Call	Supervisor
0x0C	Prefetch Abort	Abort
0x10	Data Abort	Abort
0x14	Not Used	NA
0x18	IRQ Interrupt	IRQ
0x1C	FIQ Interrupt	FIQ

- Ripristino dei registri

L'ARM e il MIPS sono architetture load/store ovvero solo queste istruzioni possono accedere alla memoria, mentre il motorola ha diverse istruzioni in grado di farlo. Inoltre l'arm e il MIPS sono a lunghezza fissa (hanno la pipe).

L'ARM e il MIPS usano uno specifico registro per salvare l'indirizzo di ritorno di una subroutine, mentre il MC68K lo salva sullo stack.

2. Overview dei sistemi cloud, IoT, edge, e fog

2.1 Cloud Computing

Il cloud computing racchiude un insieme di tecnologie che permettono di compiere operazioni di elaborazione, networking e memorizzazione/archiviazione di dati utilizzando risorse hardware/software distribuite e virtualizzate.

La virtualizzazione delle risorse consente di astrarle dalla propria base fisica e di fornirle all'utente finale come servizi in maniera on-demand e scalabile.

Qual è la Differenza tra comprare un sistema e prendere un servizio cloud?

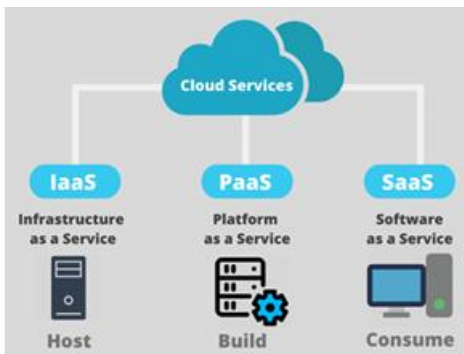
Se compro degli oggetti, quelli fanno parte del mio patrimonio, un giorno li posso rivendere. Invece se acquisto un servizio non è mio quel bene, non appartiene al mio patrimonio. I soldi per i servizi fanno parte della spesa corrente. Se non abbiamo tanti soldi per i servizi il cloud è un problema. Invece i soldi per gli acquisti di oggetti appartengono ad altri fondi. I servizi vanno su altri capitoli di spesa. L'acquisizione di un bene di ingegneria è complessa.

Le risorse sono disponibili in rete e sono accessibili tramite meccanismi standard. Le risorse sono distribuite dal provider dinamicamente a più utenti (resource pooling).

Le risorse inoltre possono essere fornite e rilasciate in modo elastico così da scalare molto rapidamente. Inoltre c'è un sistema di monitoraggio dell'uso delle risorse.

- Nel modello classico di gestione dei servizi IT (on-premise), un'azienda acquisisce e gestisce in-house le soluzioni hardware e software necessarie al proprio business -> L'azienda è proprietaria dell'hardware e delle licenze del software che utilizza. Il personale dell'azienda si occupa dell'installazione, configurazione e manutenzione.
- Nel modello cloud le risorse sono all'interno dell'infrastruttura di proprietà del provider e le aziende possono accedervi in qualsiasi momento attraverso piattaforme accessibili via web. I servizi cloud possono essere erogati secondo diversi **Modelli di servizio** e di **fornitura**.

2.2 Modelli di Servizio



riguarda in che modalità un servizio può essere offerto.

- 1) **SaaS (software as a service):** acquisizione di intere applicazioni software in esecuzione su infrastrutture Cloud e accessibili da remoto via Web. Esempi sono la posta elettronica, i calendari oppure gli strumenti di produttività come Microsoft Office. Il provider se ne assume le responsabilità in termini di sicurezza. Non è necessario sostenere una spesa impegnativa per l'acquisto del software.
- 2) **IaaS (infrastructure as a Service):** acquisizione di una infrastruttura virtuale scalabile al di sopra delle quali eseguire le proprie applicazioni come i data center. Si eliminano le spese di capitale grazie al modello pay-per-use.
- 3) **PaaS (Platform as a service):** acquisizione di ambienti e tool web-based per lo sviluppo, il testing, l'hosting e la manutenzione delle proprie applicazioni.

L'architettura di riferimento definita dal NIST prevede 5 attori:

- Cloud consumer: utilizza servizi offerti da Cloud providers.
- Cloud provider: è responsabile di rendere servizi disponibili a parti interessate.
- Cloud auditor: conduce una valutazione indipendente dei servizi cloud, delle operazioni del sistema informativo, delle prestazioni e della sicurezza del sistema Cloud.
- Cloud broker: gestisce l'uso, le performance e la distribuzione dei servizi cloud; negozia le relazioni tra provider e consumer.
- Cloud carrier: intermediario che fornisce connettività e trasporto dei servizi cloud, da provider e consumer.

2.3 Modelli di Fornitura

Riguarda il modo in cui vengono forniti i servizi

- **Cloud Privato:** l'infrastruttura Cloud può essere ospitata nei data center del provider oppure gestito da una terza parte, ma viene fornito per l'uso esclusivo di una singola organizzazione. Il modello privato tende ad essere preferito per l'elaborazione e memorizzazione delle informazioni più sensibili, poiché lascia al customer il controllo dei propri dati e processi.
- **Cloud pubblico:** l'infrastruttura è ospitata nei data center di un provider Cloud ed è condivisa fra customer diversi. Riduce drasticamente i costi, perché il provider ammortizza le spese di gestione su milioni di utenti. Sono preferite per l'esecuzione di task a basso livello di priorità.
- **Cloud ibrido:** utilizzo dei due modelli ibrido.

Perché si fa il cloud e non si comprano sistemi? Chi garantisce questa filosofia? Non è una scelta arbitraria del singolo ente, ma un obbligo normativo. La strategia nazionale è dettata dall'**AGID (Agenzia per l'Italia Digitale)**, che attraverso il *Piano Triennale per l'Informatica nella PA* ha istituito il principio del "**Cloud First**". Questo significa che le PA, per legge, prima di

acquistare nuovi server fisici devono valutare e preferire l'adozione di servizi Cloud. Nessuno può contestare questa spesa perché è l'attuazione di una direttiva nazionale.

Il ruolo di ACN (Agenzia per la Cybersicurezza Nazionale) e la paura sui dati La tua preoccupazione su "dove sono i dati e chi li usa" è esattamente il motivo per cui entra in gioco l'ACN. La direttiva più temuta (e cruciale) riguarda la **Sovranità dei Dati** e la classificazione delle informazioni. L'ACN stabilisce che i dati critici e strategici dello Stato devono risiedere su infrastrutture sicure (spesso sul territorio nazionale o europeo). L'ACN qualifica i provider: una PA non può comprare un cloud qualsiasi, ma solo quelli certificati e presenti nel catalogo nazionale dei servizi cloud qualificati. Questo si ricollega direttamente ai rischi di sicurezza e "loss of governance" citati nelle slide.

Consip e MEPA: Come si compra il Cloud?

- **Consip:** È la centrale acquisti della PA. Senza Consip, ogni singolo comune o ente dovrebbe fare una gara d'appalto complessa per valutare la sicurezza e i costi dei provider cloud. Consip invece stipula degli *Accordi Quadro* a monte. Seleziona i fornitori certificati e fissa i prezzi.
- **MEPA (Mercato Elettronico della PA):** È letteralmente l'"e-commerce" della PA gestito da Consip. Sì, i servizi Cloud (IaaS, PaaS, SaaS) si comprano sul MEPA. L'ente pubblico entra nel portale e "acquista" il servizio attingendo ai fornitori già validati.

Il problema dei fondi: Patrimonio vs Servizio (CAPEX vs OPEX) Questa è un'osservazione centratissima. In economia aziendale si traduce nella differenza tra **CAPEX** (Spese in conto capitale / Investimenti) e **OPEX** (Spesa corrente / Operativa):

- **On-premise (Comprare server):** È un investimento (CAPEX). I server diventano patrimonio dell'ente. Vengono pagati con fondi di investimento.
- **Cloud (SaaS/IaaS/PaaS):** È un abbonamento mensile/annuale (OPEX). Si paga in base all'utilizzo effettivo. Grava sulla *spesa corrente* dell'ente.

Il Paradosso del PNRR: Il PNRR finanzia l'innovazione e la migrazione al cloud (che è visto come un investimento iniziale). Ma una volta finito il PNRR, l'ente deve pagare l'abbonamento cloud ricorrente usando i propri fondi di *spesa corrente*, che specialmente per i comuni del Sud o i piccoli enti sono molto limitati. Hai fatto l'esempio perfetto: costruisci l'impianto con i fondi europei, ma poi non hai i soldi per la manutenzione e la gestione ordinaria.

In generale, affidandosi al Cloud si perde (totalmente o parzialmente, a seconda del modello di servizio e di fornitura scelto) il controllo dei propri dati e sistemi, e ci si espone a dei rischi di sicurezza che dipendono sia dalle misure di sicurezza implementate dall'utente nelle proprie applicazioni che dalle politiche di protezione implementate dal fornitore del servizio Cloud

L'ENISA stabilisce dei principali rischi di sicurezza:

- Utilizzando le infrastrutture cloud, i consumatori cedono parte dei controlli al cloud
-

3 IoT

"non è semplicemente il singolo "oggetto smart", ma è l'**intera infrastruttura globale** che permette alla società di avere **servizi avanzati**. Lo fa mettendo in rete e facendo comunicare tra loro "cose" (che siano

dispositivi fisici reali o rappresentazioni virtuali), sfruttando le tecnologie di comunicazione attuali e in via di sviluppo”

I dispositivi coinvolti in un Sistema IoT possono essere anche molto eterogenei in quanto a capacità di elaborazione e memorizzazione, connettività e alimentazione.

Il vero valore dell'IoT nasce quando molti dispositivi fisici dialogano tra loro e con i sistemi centrali (come il Cloud) per automatizzare processi e prendere decisioni.

Si sviluppa su 3 livelli:

- **livello dei dispositivi:** comprende sensori, attuatori e sistemi embedded in grado di generare tipicamente grosse moli di dati.
- **Livello di connettività:** include i protocolli che consentono ai dispositivi di comunicare con il livello dei servizi e applicazioni.
- **Livello dei servizi e applicazioni:** solitamente include servizi Cloud che si occupano di elaborare i dati generati dai dispositivi.

I dispositivi IoT sono insicuri per natura: sono connessi e quindi potenzialmente accessibili, spesso non posseggono neanche le capacità per proteggersi. Essi sono spesso soggetti ad attacchi di tipo Ransomware, denial of service, furto di identità e di dati che possono avere effetti negativi in termini di security, privacy e safety.

4 Edge Computing

Edge computing significa elaborazione ai margini ovvero elaborazione delle informazioni in prossimità della relativa sorgente.

Nasce per rispondere all'esigenza di ridurre la latenza e il consumo di banda che caratterizzano i sistemi IoT in cui grossi moli di dati vengono inviati a servizi cloud. Inoltre si riducono i rischi di sicurezza legati alla privacy.

I componenti di un sistema Edge sono sicuramente i devices ovvero i dispositivi attrezzati per raccogliere dati ed eseguire elaborazioni in locale.

Ci sono poi i server perimetrali che vengono utilizzati per distribuire le applicazioni sui dispositivi.

Poi ci sono dei microdatacenter che possono essere visti come un Cloud Locale con cui i dispositivi possono comunicare.

5 Fog Computing

Il concetto di Fog computing è molto simile al concetto di edge computing. I Fog Node sono tipicamente schierati nelle posizioni intermedie tra i nodi edge e i servizi cloud.

Le loro capacità di calcolo sono nettamente superiori degli edge, consentendo l'hosting di servizi di virtualizzazione, di integrazione di vari dispositivi edge tramite la conversione di protocolli di networking.

Un Fog Node è in grado di raccogliere dati provenienti da molteplici e differenti nodi Edge circostanti, traducendo e convertendo i diversi protocolli di rete per uniformare le informazioni prima di un eventuale inoltramento verso il Cloud.

